

Class 3

Deploy Static Web

*Today you put something on the internet that other people can load.
Almost everything that goes wrong today goes wrong the same way in every class after this one.*

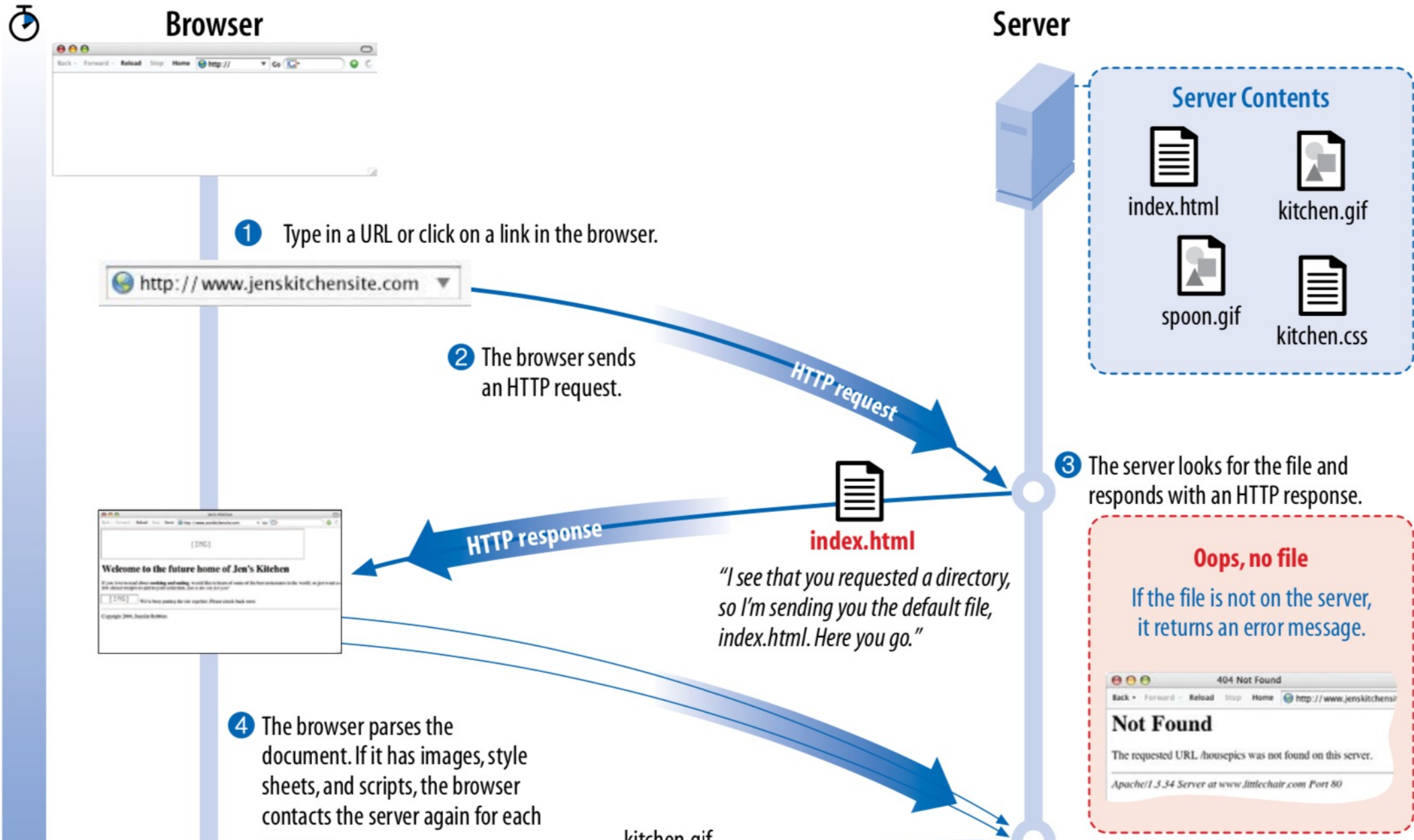
Where we have got to

THE SHAPE OF THIS COURSE

- Class 1 — packages. Software arrives from somewhere, and something decides where
- Class 2 — processes and services. A program that survives you logging out
- Class 3 — a web server, serving files, reachable by somebody who is not you
- Class 4 — a program and a database behind it.
- Class 5 — the two joined together.
- The same application all three weeks, and again in containers from class 6.

You are deploying software you did not write. That is the normal case, and it is the skill.

How browsers display web pages



1

Where your software comes from

apt install nginx does not install nginx. It installs whichever nginx this machine has been told about.



nginx [engine x] is an HTTP and reverse proxy server, a mail proxy server, and a generic TCP/UDP proxy server

What is another popular static web server software?

There is more than one nginx

PART 1 · REPOSITORIES

- The distribution packages one build. The people who write nginx package another.
- Different version numbers, different default configuration, *different file layouts.
- Every tutorial you find assumes one of the two, and never says which.
- We use the upstream build — it is the same one inside the official container image, which you meet in class 6. Learn one layout, not two.

`apt-cache policy <pkg>` tells you which versions exist and, more usefully, where each of them comes from.

nginx on Ubuntu main repo

```
$ ls /etc/nginx
conf.d          koi-utf      modules-available  proxy_params  sites-enabled  win-utf
fastcgi.conf    koi-win      modules-enabled    scgi_params   snippets
fastcgi_params  mime.types   nginx.conf         sites-available uwsgi_params
```

```
$ cat /etc/nginx/nginx.conf
user www-data;
include /etc/nginx/conf.d/*.conf;
        include /etc/nginx/sites-enabled/*;
```

```
$ cat /etc/nginx/sites-available/default
root /var/www/html;

    index index.html index.htm index.nginx-debian.html;
    server_name _;
    location / {
        try_files $uri $uri/ =404;
    }
```

```
$ ls /etc/nginx
conf.d  fastcgi_params  mime.types  modules  nginx.conf  scgi_params  uwsgi_params

$ cat /etc/nginx/nginx.conf
user nginx;
    include /etc/nginx/conf.d/*.conf;

$ cat /etc/nginx/conf.d/default.conf
server {
    listen      80;
    server_name localhost;

    #access_log  /var/log/nginx/host.access.log  main;

    location / {
        root    /usr/share/nginx/html;
        index   index.html index.htm;
    }
}
```


Adding a repository, safely

PART 1 · TRUST

```
$ curl -fsSL <key-url> | sudo gpg --dearmor -o /usr/share/keyrings/<name>.gpg

$ cat /etc/apt/sources.list.d/<name>.list
deb [signed-by=/usr/share/keyrings/<name>.gpg] <repository-url> <codename> <component>
```

- The key is the whole of “do I trust this software?”. Packages are signed; apt refuses anything the key does not vouch for.
- **signed-by=** binds ONE key to ONE repository
- Without it the key is trusted for every repository on the machine — which is how a compromised third party ends up replacing your kernel.

You are about to run their code as root. This is the step that decides whether that is reasonable.

2

From installed to running

Installing puts files on a disk. Everything class 1 and 2 taught about enabled and active now applies to something strangers can connect to.

Installed is not running

PART 2 · SERVICES

- `systemctl is-enabled` — will it come back after a reboot?
- `systemctl is-active` — is it running right now?
- Two independent states. You met them with cron in class 1 and your own service in class 2.
- A package MAY start what it installs, and MAY enable it, and the two are separate decisions its packager made.
- Find out what yours decided before you touch it. Then look again after.

`systemctl status` shows both, plus the main pid and the last few log lines.

One service, several processes

PART 2 · PRIVILEGE

```
$ ps -o user,pid,ppid,cmd -C nginx
USER      PID      PPID  CMD
?         264545      1 nginx: master process ...
?         264546  264545 nginx: worker process
?         264547  264545 nginx: worker process
```

- One master, several workers, and *they do not all run as the same user.
- Ports below 1024 may only be bound by a privileged process — so the master needs privilege, once, at startup.
- The workers are the ones that read your files and talk to strangers. They do not keep it.
- Go and fill in that first column yourself. When a file of yours turns out to be unreadable in class 4, this is why.

This is the service class 2 promised you: one that deliberately does not run everything as the same user.

3

Listening is not the same as reachable

The part of today that matters most. Every remaining class deploys something on a port, and this failure looks identical every time.

Three things stand between your file and a visitor

PART 3 · THE TAXONOMY OF FAILURE

1. The process — is the server running at all? `systemctl is-active`
 2. The port — is it listening, and on which addresses? `ss -tln`
 3. The path in — is anything between you and the visitor dropping the packet?
- Three separate questions. `curl localhost` answers the first two and tells you NOTHING about the third.
 - Every one of them fails differently, and the difference is the diagnosis.

`0.0.0.0` means every address on the machine. Compare that with a service listening only on 127.0.0.1 — that distinction returns in class 5.

Predict, then go and find out

PART 3 · DO NOT SKIP THIS

```
# on the VM
$ ss -tln          → your server is listening on :80
$ curl localhost   → a page comes back

# now, from your own laptop, in a browser
http://<your-vm>.sit.kmutt.ac.th/
```

- Before you run it: what do you think happens? Say it out loud.
- Nothing about the server changes between those two requests. Same process, same port, same page.
- Whatever happens, notice the exact way it fails. A refusal and a silence are not the same thing.

I am not going to run this for you. Steps 12 to 14 are worth more to you than watching me do it.

ufw (uncomplicated firewall) provides easy-to-use interface to manage firewall

status	show firewall status
status numbered	show firewall status as numbered list of RULES
status verbose	show verbose firewall status
show ARG	show firewall report
allow ARGS	add allow rule
app list	list application profiles
deny ARGS	add deny rule
delete RULE NUM	delete RULE

```
sudo ufw allow 8080/tcp
sudo ufw status numbered
sudo ufw delete 6
sudo ufw delete allow 8080/tcp
sudo ufw show listening
```

Do not deny or delete SSH rule!

- `cat /var/log/nginx/access.log`
- `cat /var/log/nginx/error.log`

4

Serving your own files

A web server serves the directory it was told about. By default that is neither yours nor anywhere near your home directory.

- <https://www.digitalocean.com/community/tutorials/how-to-install-nginx-on-ubuntu-22-04>
- <https://www.digitalocean.com/community/tutorials/understanding-nginx-server-and-location-block-selection-algorithms>
- https://nginx.org/en/docs/http/request_processing.html
- https://nginx.org/en/docs/http/server_names.html
- <https://docs.nginx.com/nginx/admin-guide/web-server/web-server/>
- https://nginx.org/en/docs/http/nginx_core_module.html#listen

- Nginx has one server block configured in `/etc/nginx/conf.d/default.conf`

```
server {  
    listen      80;  
    server_name localhost;  
  
    #access_log  /var/log/nginx/host.access.log  main;  
  
    location / {  
        root    /usr/share/nginx/html;  
        index   index.html index.htm;  
    }  
  
    #error_page  404              /404.html;  
  
    # redirect server error pages to the static page /50x.html  
    #  
    error_page   500 502 503 504  /50x.html;  
    location = /50x.html {  
        root    /usr/share/nginx/html;  
    }  
}
```

each directive must end with ';'
the port to listen to; add 'default_server' to set as default server
hostname to serve, specify multiple hostnames separated with space
_ is invalid domain name; may use \$hostname; "" for empty host header
request URIs path to serve files
root directory of the documents
name of the file to serve when directory is specified

PART 4 · CONFIGURATION

```
server {  
    listen      80;  
    server_name localhost;  
    root  /usr/share/nginx/html;      # the directory it serves  
    index index.html;                # what to send for a directory  
    location / { try_files $uri $uri/ =404; }  
}
```

- Configuration lives in `/etc/nginx/conf.d/`, and *only files ending `.conf` are read — which makes renaming the cheapest possible off switch.
- The package ships an example server block of its own. Look at what is already in that directory before you wonder why yours is being ignored.
- Two server blocks may claim the same port quite legally. nginx does not merge them and does not complain — it picks ONE, per request.

A server block (1)

PART 4 · THE RULE

```
http {  
    include /etc/nginx/conf.d/*.conf;    # in FILENAME SORT ORDER  
}  
  
server { listen 80;                server_name shop.example; ... }  
server { listen 80 default_server; server_name class.int134; ... }
```

- Only `*.conf`, and in the order the filenames sort — not the order you created them.
- Per request, nginx compares the browser's `Host`: header against every `server_name` on that port. A match answers.
- Nothing matched? Then the DEFAULT SERVER answers — and unless you say otherwise, that is simply the FIRST block read for that socket. A filename decided which of your blocks a stranger reaches.

A server block (2)

PART 4 · THE RULE

```
http {  
    include /etc/nginx/conf.d/*.conf;    # in FILENAME SORT ORDER  
}  
  
server { listen 80;                server_name shop.example; ... }  
server { listen 80 default_server; server_name class.int134; ... }
```

- Which is why `/etc/*.d` directories the world over are full of `0_`, `10-`, `99-` prefixes. Sorting is load-bearing.
- `listen 80 default_server;` says it out loud, and the filename stops mattering. Prefer saying what you mean to arranging for it.

We do this in the room: write a block, watch the wrong page come back, and win it twice — once by sorting, once by intent.

- nginx will match listening (NIC) ip_address/port first
 - A block with no listen directive uses the value 0.0.0.0:80
 - A block with no port uses port 80 as default
 - A block with no ip_address uses 0.0.0.0 as default
- nginx find exact match, leading wildcard match, trailing wildcard match, regular expression match, in order
- If no 'default_server' keyword is specified, nginx uses the first virtual server block as the default_server
- If there is only one server block listen on the same port, it will not check the server_name

- If there are multiple server blocks listen on the same port, and the request's header field "Host" does not match any server name, it will route to the `default_server`
- If there is a conflict in server name, it will ignore the later config file (based on site configuration file name). [the use of `default_server` does not fix conflict name]
- `listen [::]:80` is for ipv6
- Empty "" will match request with no header

Two different tests

PART 4 · DOES IT LOAD, DOES IT WORK

```
$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

- `nginx -t` asks whether the configuration would LOAD. It does not touch the running server.
- When it fails it names the file, the line, and a word in brackets that tells you how bad it is. The error is logged in `/var/log/nginx/error.log`
- A configuration that loads is not a configuration that works: point root at a directory that does not exist and `nginx -t` is perfectly happy.
- Run it before every reload. A reload with a broken file keeps the old one; a RESTART with a broken file leaves you with no web server.

Get into the habit now. In class 5 you will be editing this file while a database depends on it.

The application, and its configuration

PART 4 · WHAT YOU DEPLOY

```
frontend/    index.html  app.js  store.js  style.css  data/  
              config.example.js  →  config.js
```

config.js is NOT in the repository. It is different on every machine.

- You are deploying an application you did not write and are not expected to read.
- Everything that differs between one deployment and the next lives in one small file that is not in git.
- The committed `.example` beside it tells you what to write. Nothing else in the repository knows which machine it is on.
- You meet this again in class 4 with a database password in it, and in class 8 it becomes an environment variable.

Fork the repository to your own account and clone your fork. From class 4 it holds your configuration, and in class 10 it is what deploys itself.

What a file server cannot do

PART 4 · THE POINT OF TODAY

- The page has a form. Fill it in, press the button, and the server will refuse.
- Nothing is broken. A web server hands out files that already exist.
- It has nowhere to put a new one, and no idea what it would mean to try.
- It tells you so with a specific status code — read it, and remember it, because it is the argument for next week.
- Your site is finished. Your application is not.

Class 4: the program that accepts the notice, and the database that keeps it.

Before you leave today

THE FOUR THAT MATTER

- Your site loads from a machine that is not your VM.
- You can say which of the three failures you hit, and how you knew.
- You ran `nginx -t` before a reload at least once, and you saw it fail at least once.
- You pressed Add, and you understand why it refused.
- If your site works from the VM and not from your laptop, you have not finished — that is the lab, not a technicality.